

LoxBerry Text2Speech (T2S)

Developer Guide for the MQTT Plugin Interface

MQTT Plugin Interface for LoxBerry

Current interface documentation for plugin developers.

Important: Text2Speech uses only the MQTT broker configured in LoxBerry. The T2S subscriber runs as a dedicated systemd service and processes requests on tts-publish/#.

1. Overview

The Text2Speech plugin provides other plugins with a validated MQTT interface for generating TTS files. Other plugins publish a JSON request to the standard LoxBerry MQTT broker. The continuously running T2S subscriber validates the request, generates the MP3 file or loads it from the cache, and publishes the response to the requested return channel.

- The existing LoxBerry MQTT configuration remains authoritative.

2. Runtime Components

Component	Path / Name	Purpose
Subscriber	/opt/loxberry/bin/plugins/text2speech/mqtt/mqtt-subscribe.php	Validation, TTS generation, cache, and MQTT response
systemd service	mqtt-service-tts.service	Starts the subscriber as user/group loxberry and restarts it if it terminates
Broker	LoxBerry MQTT Broker	Transports request and response topics; it is not reconfigured by the T2S plugin

3. MQTT Topics and Return Channel

The subscriber listens exclusively on:

```
tts-publish/#
```

A request topic containing the plugin and correlation identifier is recommended:

```
tts-publish/<client>/<corr>
```

For the response, each request should specify a unique reply_to:

```
tts-subscribe/<client>/<corr>
```

If reply_to is set, T2S uses this topic exactly. If reply_to is missing, T2S falls back to tts-subscribe/<client>. If client is also missing, the fallback is tts-subscribe/text2sip.

Note on corr: The corr field is still allowed as a string, but the current T2S subscriber does not use it for response correlation and does not mirror it in the success response. For unambiguous correlation, use reply_to with a unique topic.

4. Request Payload and Validation

The payload must be valid JSON. Unknown keys are rejected. At least text or function must be present with a non-empty value.

Field	Type	Required	Meaning
text	string	text OR function	Text to be spoken.
function	string	text OR function	Add-on function; see section 5.
nocache	number	no	0 = use cache, 1 = force/regenerate MP3.
logging	number	no	1 = include T2S log lines in the success response.
mp3files	number	no	1 = include a list of available MP3 files.
client	string	no	Identifier of the requesting plugin; also used as fallback for the response topic.
instance	string	no	Optional instance identifier; mirrored in original.
corr	string	no	Allowed correlation identifier; currently not evaluated by the subscriber.
reply_to	string	recommended	Exact MQTT topic for the response.

Example Request

Topic: tts-publish/myplugin/7f60f8e9

```
{
  "client": "myplugin",
  "corr": "7f60f8e9",
  "reply_to": "tts-subscribe/myplugin/7f60f8e9",
  "text": "This is a test output",
  "nocache": 0,
  "logging": 1,
  "mp3files": 0
}
```

Additional validation rules:

- **Invalid JSON is answered with status=error.**
- **Keys that are not allowed result in “Invalid keys in JSON”.**
- **Incorrect data types result in “Invalid types”.**
- **text and function must not both be empty or missing.**
- **Strings are sanitized before processing; disallowed control/special characters are removed.**
- **A byte-identical payload is ignored as a duplicate within a 25-second window.**

5. Function Calls

Instead of static text, function can be used. The subscriber calls the corresponding T2S add-on function and then passes the generated text to the normal TTS generation process.

function	Meaning
weather	Weather text
clock	Time
warning	Weather warning

pollen	Pollen forecast
abfall	Waste/garbage calendar

Example Function Request

```
{
  "client": "myplugin",
  "corr": "7f60f8e9",
  "reply_to": "tts-subscribe/myplugin/weather-001",
  "function": "weather",
  "logging": 1
}
```

6. Success Response

After successful generation or a successful cache hit, T2S publishes JSON with status=done to the determined response topic. The actual URL depends on the T2S system configuration.

```
{
  "status": "done",
  "message": "MP3 file created successfully.",
  "file": "58439455ca9189d392a8f819be70ef5d.mp3",
  "httpinterface": "http://<IP-ADDRESS OF TEXT2SPEECH PLUGIN>/plugins/text2speech/interfacedownload",
  "httpmp3interface": "http://<HOSTNAME OF TEXT2SPEECH PLUGIN>/plugindata/text2speech/mp3",
  "cifsinterface": "://<HOSTNAME OF TEXT2SPEECH PLUGIN>/plugindata/text2speech/interfacedownload",
  "ttspath": "/opt/loxberry/data/plugins/text2speech/tts",
  "mp3path": "/opt/loxberry/data/plugins/text2speech/mp3",
  "cifsmmp3interface": "://loxberry/plugindata/text2speech/mp3",
  "timestamp": "17:32:10",
  "corr": "1761755529",
  "interfaces": {
    "httpinterface": "http://<IP-ADDRESS OF TEXT2SPEECH PLUGIN>/plugins/text2speech/interfacedownload",
    "httpmp3interface": "http://<HOSTNAME OF TEXT2SPEECH PLUGIN>/plugindata/text2speech/mp3"
  },
  "original": {
    "corr": "1761755529",
    "reply_to": "tts-subscribe/text2sip/1761755529",
    "client": "text2sip",
    "instance": null
  },
  "mp3files": [
    "100_Tuerklingel 3x.mp3",
    "17_schulglocke_2x.mp3",
    "18_schulglocke_3x.mp3",
    "1_Alarmsirene.mp3",
    "2_Airport_gong.mp3",
    "3_Ding-noise.mp3",
    "4_Old-fashioned-doorbell.mp3"
  ],
  "logs": [
    "<INFO> Payload received from [tts-publish/text2sip/1761755529]:",
    {"corr\":\"1761755529\", \"client\": \"text2sip\", \"nocache\": 1, \"mp3files\": 1, \"logging\": 1, \"text\": \"hallo, Dies ist ein erneuter Testanruf für Nele\", \"reply_to\": \"tts-subscribe/text2sip/1761755529\"},
    "<OK> Valid JSON received. Processing TTS request...",
    "<INFO> nocache=1 -> re-create MP3: /opt/loxberry/data/plugins/text2speech/tts/58439455ca9189d392a8f819be70ef5d.mp3",
    "<OK> MP3 file re-created successfully.: 58439455ca9189d392a8f819be70ef5d.mp3"
  ]
}
```

If mp3files=1 is set, the response additionally contains mp3files. If logging=1 is set, it additionally contains logs.

7. Error Responses

Validation or runtime errors are published to the currently available response topic with status=error.

```
{
  "status": "error",
  "message": "Invalid keys in JSON",
  "details": {
    "invalid_keys": ["nocach"],
    "original": { "nocach": 1 }
  },
  "logs": [],
  "timestamp": "2026-08-14T08:00:00+02:00"
}
```

Typical error messages are “Invalid JSON syntax”, “Invalid keys in JSON”, “Missing mandatory field: text or function”, “Invalid types”, “Invalid function value”, “Directory not found”, “Directory not writable”, and “MP3 could not be created”.

8. Recommended Flow for Calling Plugins

1. Obtain broker connection data from the standard LoxBerry MQTT configuration.
2. Generate unique client and corr values.
3. Build a unique reply_to topic, e.g. tts-subscribe/myplugin/<corr>.
4. Subscribe to exactly this response topic first.
5. Then publish the JSON payload to tts-publish/myplugin/<corr> (QoS 0 is sufficient).
6. Receive the response on reply_to and check status.
7. If status=done, use the returned file or the configured HTTP/CIFS interface.
8. Close or reuse the subscription depending on the plugin architecture.

Minimal Perl Example

```
use LoxBerry::IO;
use Net::MQTT::Simple;
use JSON qw(encode_json decode_json);

my $client = "myplugin";
my $corr = time();
my $req = "tts-publish/$client/$corr";
my $resp = "tts-subscribe/$client/$corr";

my $c = LoxBerry::IO::mqtt_connectiondetails();
my $mqtt = Net::MQTT::Simple->new("$c->{brokerhost}::$c->{brokerport}");
$mqtt->login($c->{brokeruser}, $c->{brokerpass}) if $c->{brokeruser} || $c->{brokerpass};

$mqtt->subscribe($resp => sub {
  my ($topic, $message) = @_;
  my $answer = decode_json($message);
  # evaluate status / file
});

$mqtt->publish($req, encode_json({
  client => $client,
  corr => "$corr",
  reply_to => $resp,
  text => "Test output",
  nocache => 0
}));
```

9. Service and Troubleshooting

The T2S subscriber is managed by systemd. The most important checks are:

```
systemctl status mqtt-service-tts.service
journalctl -u mqtt-service-tts.service -n 50 --no-pager
systemctl status mosquitto
```

The service starts `/opt/loxberry/bin/plugins/text2speech/mqtt/mqtt-subscribe.php` as user and group `loxberry`. If it terminates, it is automatically restarted after 2 seconds.

- If the subscriber cannot connect to the broker, it retries the connection.
- After a connection loss, `tts-publish/#` is subscribed again after reconnecting.

10. Security and Integration Rules

- Security and authentication are governed exclusively by the central LoxBerry MQTT configuration.
- Calling plugins should not create their own Mosquitto listeners or T2S-specific broker configurations for T2S.
- The T2S plugin does not require any changes to `/etc/mosquitto` for this interface.
- Always use unique `reply_to` topics for parallel requests.
- Send only documented JSON keys; `additionalProperties` is disabled and unknown keys are rejected.